Mastering Claude Code

From Foundations to Agentic Mastery

Learning Path

Module 0 — Setup

- 01 — Account Registration
- 02 — Claude Code Installation

Module 1 — Foundation

- 01 — What is Claude Code?
- 02 — Tokens and Context Window
- 03 — Interacting with Agentic AI (PACT Framework)
- 04 — Claude Code Extensions
- 05 — Module Exercises

Module 2 — Core Skills

- 01 — Prompt Engineering
- 02 — Writing Specs
- 03 — Module Exercises

Each phase builds on the previous — mental models first, daily tools second, advanced features last.

Learning Path

← *Continuing from Module 0, 1 & 2*

Module 3 — Governance

- 01 — AI-Assisted Development Workflow
- 02 — Sample Usage of AI-Assisted Development Workflow
- 03 — AI Usage Policy

Module 4 — Capstone

- 01 — Capstone Project

🎓 *Everything you've learned — applied end-to-end in a real project.*

Each phase builds on the previous — mental models first, daily tools second, advanced features last.

MODULE 00

Installation & First Session



From team invitation to your first Claude Code session — step by step.

What you'll do:

1. Check email
2. Setup Claude.ai
3. Install Git CLI
4. Install Claude Code

Internal Use Only – Do Not Share Outside the Organization

Step 1 — Accept Invitation & Set Up Claude.ai

Check your email invitation

From: noreply@anthropic.com
Subject: You've been invited to a Claude Team plan

Hi [Your Name],

You've been invited to join your organisation's Claude Team plan. Click the button below to accept and create your account.

Accept Invitation →

Link expires in 7 days. Check spam if not received.

- ! Check your spam folder if not in inbox within 5 minutes
- ! Use your work email — must match the address your manager registered
- ! The link is single-use. Ask your admin to resend if it expires

Set up your Claude.ai account

1 Click the invitation link

Opens claude.ai. Choose 'Create account' if you're new, or 'Sign in' if you already have a personal account.

2 Complete account setup

Set your name and password. Verify your email if prompted. Two-factor auth is recommended for team accounts.

3 Confirm you're on the Team plan

After login, go to Settings → Plan. You should see your organisation's Team plan — not the free tier.

4 Try a quick chat first

Before installing Claude Code, send one message in Claude.ai to confirm your account works and you're authenticated.

Internal Use Only – Do Not Share Outside the Organization

Step 2 — Install Git CLI for Windows

Why Git CLI is required

`git diff`

Claude reads your uncommitted changes before reviewing or modifying code

`/commit`

Claude creates git commits — git must be in PATH for this to work

`/review`

Requires git to compare staged vs unstaged changes in a PR

`Safety net`

Always commit before a large agentic run — instant rollback if something goes wrong

`/diff`

Built-in command to view all changes Claude made per turn — needs git

Install Git for Windows

1 Download Git CLI

Go to git-scm.com/download/win and download "64-bit Git for Windows Setup"

2 Run the installer

Accept all defaults. On the PATH screen: select "Git from the command line and also from 3rd-party software"

Windows Terminal

```
git --version
git version 2.47.1.windows.2  ✓
```

First-time Git setup (run once)

Set your name and email so commits are attributed correctly:

```
git config --global user.name "Your Name"
git config --global user.email "you@company.com"
```

Internal Use Only – Do Not Share Outside the Organization

Step 3 — Install Claude Code on Windows

1

Open an Admin terminal

Press Win + X → select Windows Terminal (Admin) or Command Prompt (Admin).

Run as Administrator is required for the global install to succeed.

2

Run the install command

Copy and paste this single command:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd &&
install.cmd && del install.cmd
```

Downloads, installs, then deletes the installer automatically.

3

Authenticate

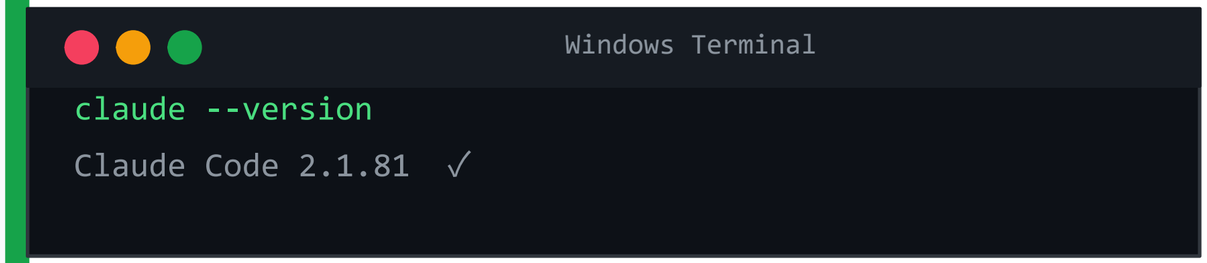
A browser tab opens automatically. Sign in with the Anthropic account you set up in Step 1. The CLI saves your token.

What that command does

`curl -fsSL …`	Downloads install.cmd from claude.ai
`-o install.cmd`	Saves it locally as install.cmd
`&& install.cmd`	Runs it (installs Node.js + Claude Code globally)
`&& del install.cmd`	Cleans up — removes installer after use

Verify installation

Open a new terminal (no admin needed) and run:



Step 4 — Add Claude Code to Your User PATH

Why this is needed: The installer places the Claude Code binary in **C:\Users\(username)\.local\bin**. Windows won't find it in a new terminal unless this folder is in your User PATH.

Adding the PATH variable — step by step

1. **Press Win and search "Environment Variables"**
   Select "Edit the system environment variables" from the Start menu results.

2. **Click "Environment Variables..." button**
   Found at the bottom of the System Properties → Advanced tab dialog.

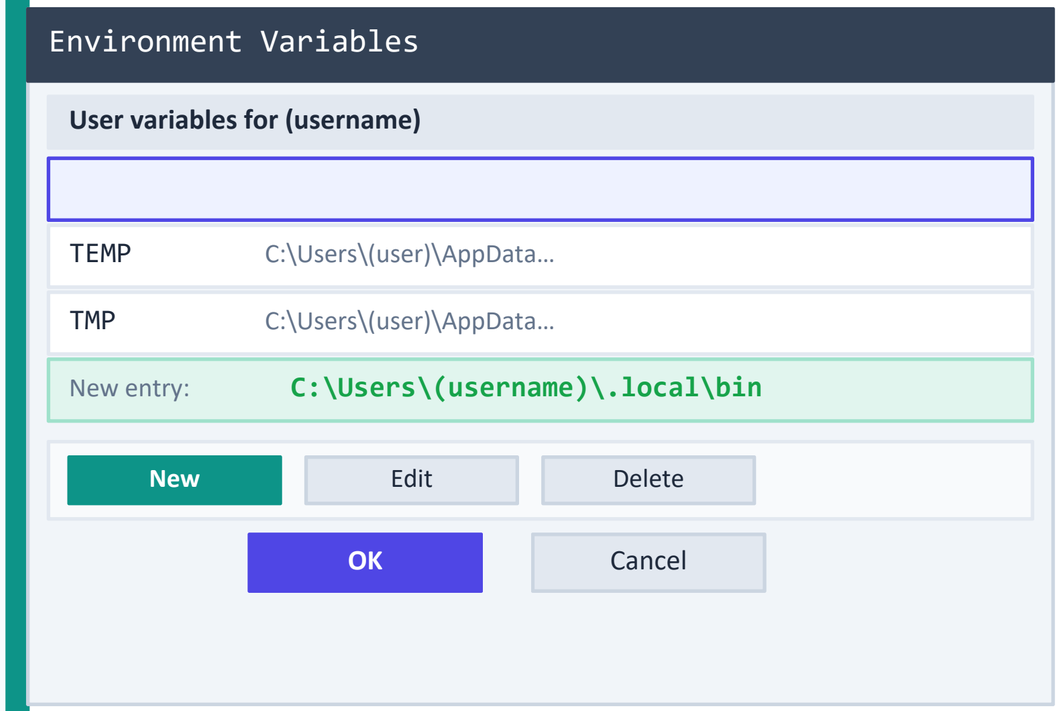
3. **Under "User variables" — select Path → Edit**
   Do NOT edit System variables — only User variables. Scroll down to find Path and double-click it.

4. **Click "New" and add the path**

```
C:\Users\(username)\.local\bin
```

5. **Click OK on all three dialogs**
   Then open a brand new terminal window. The change only applies to new terminals — not already open ones.

Environment Variables UI



Internal Use Only – Do Not Share Outside the Organization

Step 5 — Launch Your First Claude Code Session

Starting a session

1 Navigate to your project

```
cd C:\Projects\my-app
```

Always start inside your project folder. Claude reads files relative to where you launch it.

2 Launch Claude Code

```
claude
```

The CLI starts. You'll see the model name and project detected automatically.

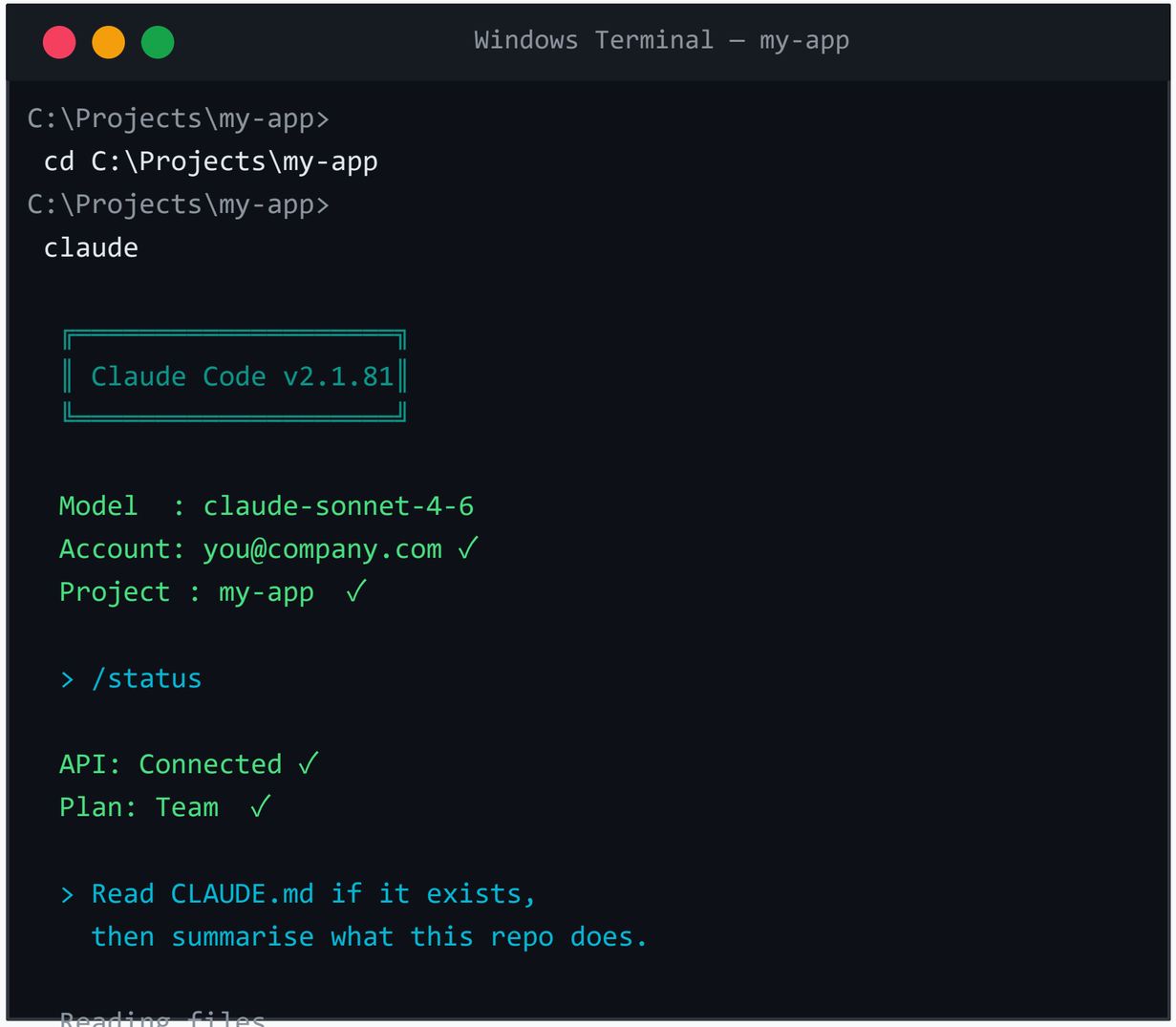
3 Check status

```
/status
```

Confirms model, your Anthropic account, and API connectivity. Always do this first.

4 Give your first task

Skip 'hello'. Go straight to a real task. Prompting is covered in the next module.



Internal Use Only – Do Not Share Outside the Organization

MODULE 01

What is Claude Code?

It is an AI-powered coding assistant developed by Anthropic. It helps developers write, understand, and improve code using natural language prompts.
Unlike traditional tools that focus solely on generating output, Claude is designed for collaboration. It can explain code, suggest improvements, and even help debug code while maintaining clarity and context.

Internal Use Only – Do Not Share Outside the Organization

Claude · Claude Code · Anthropic API



Chat

Claude.ai

Web & mobile chat interface. Consumer-focused. Great for exploration and Q&A. No local file system access.



Dev Tool

Claude Code

CLI-first agentic coding tool. Reads your files, runs commands, and operates autonomously on multi-step tasks.



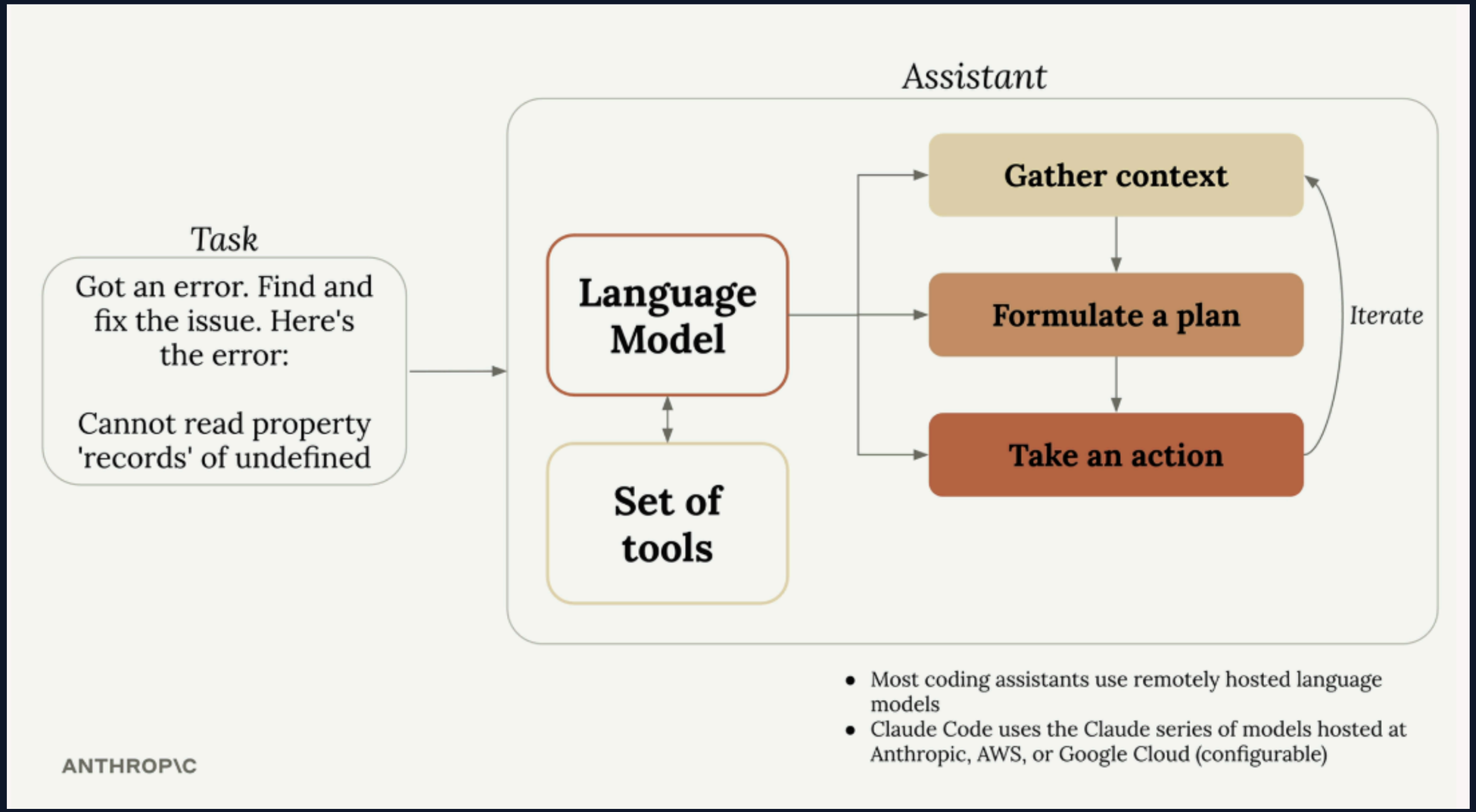
Build

Anthropic API

Programmatic REST/SDK access. Build Claude into your own apps, products, and pipelines.

Key insight: Claude is the model. The products are different access surfaces.

Coding Assistant



- Most coding assistants use remotely hosted language models
- Claude Code uses the Claude series of models hosted at Anthropic, AWS, or Google Cloud (configurable)

Internal Use Only – Do Not Share Outside the Organization

Claude Code — Tools Reference

Tool	Category	What it does	Tool	Category	What it does
`Read`	File system	Reads any file into context	`WebSearch`	Web	Searches the web for docs or errors
`Write`	File system	Creates or overwrites a file on disk	`WebFetch`	Web	Fetches and reads a specific URL
`Edit`	File system	Targeted edits to an existing file	`Task`	Agentic	Spawns a sub-agent for a discrete task
`MultiEdit`	File system	Multiple edits to one file in one op	`TodoRead`	Agentic	Reads the current session task list
`Bash`	Execution	Runs any shell command; returns stdout + stderr	`TodoWrite`	Agentic	Updates the session task list
`Glob`	File system	Finds files matching a pattern	`NotebookRead`	File system	Reads a Jupyter .ipynb notebook
`Grep`	File system	Searches file contents for a pattern	`NotebookEdit`	Edit	Adds or edits cells in a notebook
`LS`	File system	Lists directory contents	`MCP tools`	MCP	Any tool from a connected MCP server (Jira, GitHub, Notion…)

Source: Claude Code CLI v2.1.81 · code.claude.com/docs

File system | Execution | Web | Agentic | Edit | MCP

MODULE 01

The Claude Model Family

Opus, Sonnet, Haiku — three tiers, each making a different speed/cost/quality tradeoff.

Internal Use Only – Do Not Share Outside the Organization

Three-Tier Model Architecture

Opus

```
claude-opus-5-5
```

Speed: ●●○○○ Cost: $$$$$

Best for: Complex multi-file refactors, architecture decisions, nuanced reasoning where quality is critical and time is secondary.

Sonnet

```
claude-sonnet-5-5
```

Speed: ●●●○○ Cost: $$$○○

Best for: Most daily production tasks — feature implementation, test writing, documentation, code review. The safe default.

Haiku

```
claude-haiku-5-5
```

Speed: ●●●●● Cost: $○○○○

Best for: High-volume pipelines, latency-sensitive tasks, batch processing, simple generation. Alias: claude-haiku-4-5

When unsure → start with Sonnet. Upgrade to Opus for genuinely complex tasks. Drop to Haiku for volume.

MODULE 01

Tokens & Context Windows

The engine constraints that shape cost, quality, and what's possible in a session.

Internal Use Only – Do Not Share Outside the Organization

Tokens, Context Windows & Compaction

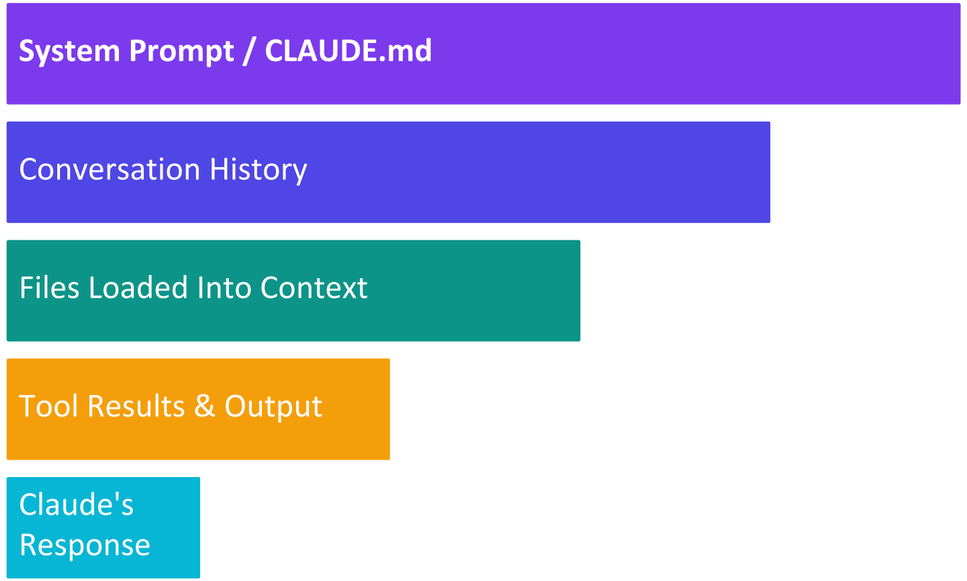
What is a Token?

A token is a subword fragment — not a character, not a word.

`"Hello"`	→ 1 token
`"Hello, world!"`	→ 4 tokens
`"Refactoring"`	→ 3 tokens
`1,000 words`	→ ~750 tokens
`100-line code file`	→ ~700 tokens

Cost rule: output tokens cost more than input tokens.

The Context Window



200K token window = ~150K words. But filling it degrades quality. Run /context to see what's using space

⚡ **Auto-compaction fires at 100,000 tokens by default.** Manual: /compact [hint] — clears history but keeps a context summary. Use /clear for a completely fresh start.

Context Window Best Practices

1 Load only what you need

Point Claude to specific files rather than entire directories. 'Read src/auth/views.py' uses far fewer tokens than 'read everything in src/'.

2 Put critical context first

Earlier tokens carry more weight in the model's predictions. Put constraints, CLAUDE.md rules, and key facts before the task description.

3 Monitor your rate limit with /usage

Run /usage to see % of your 5-hour allowance used and the exact reset time. Start every day here — it tells you how much runway you have.

```
/usage    → shows % used + reset time
```

4 Compact at milestones, not crises

Don't wait until Claude starts giving poor answers. Compact after completing a feature or PR — while context is still clean.

```
/compact [what we just built]
```

5 One task per session

Starting a fresh session for unrelated tasks is cheaper than carrying dead context. Use /clear between completely different workstreams.

```
/clear
```

6 Keep CLAUDE.md lean

Every line of CLAUDE.md loads into context on every session. Audit it regularly — remove outdated rules, merge duplicates.

⚠ Symptoms of a bloated context — if you see these, compact immediately:

- Claude ignores rules from earlier in the session
- Responses get shorter and less detailed
- Claude starts repeating the same caveats
- Earlier file changes are 'forgotten'
- Output quality degrades for no obvious reason

Rule: a 200K token window is not an invitation to fill it — quality degrades well before the limit. Use /usage daily.

MODULE 01

Interacting with Agentic AI

The PACT Framework

A native Claude Code workflow for consistent, high-quality results — every session.

P — **Plan**

/plan mode + provide context

A — **Approve**

Read, review & approve the plan

C — **Code (Execute)**

Generate based on approved plan

T — **Test**

Run tests, fix, iterate

Internal Use Only – Do Not Share Outside the Organization

The PACT Framework — Step by Step

P — Plan

Activate /plan mode and provide your full context here — stack, conventions, constraints, and the task.

Claude will think through the approach before touching any code.

→

A — Approve

Read the plan Claude generated. Check for misunderstandings, missing steps, or wrong assumptions.

Correct it if needed — then explicitly approve before moving forward.

→

C — Code

Now let Claude generate. Because the plan is approved, the output is targeted and aligned.

Go one feature or file at a time to keep context quality high.

→

T — Test

Never accept output blindly. Run the tests, paste errors back to Claude, and let it fix them.

This closes the agentic loop — gather, plan, act, iterate.

Internal Use Only – Do Not Share Outside the Organization

PACT in Practice — Real Example

Scenario: Adding a Login Feature — Python app using JWT (SimpleJWT)

P **Plan**

/plan mode

→ *"Go to /plan mode: Build a simple FastAPI backend with a /login endpoint using JWT authentication, protect routes similar to IsAuthenticated, and include a minimal HTML+JavaScript frontend to test login and access a protected endpoint"*

A **Approve**

Review output

→ *"[Read the plan] — Looks good. Step 2 should also include token refresh endpoint. Update that, then proceed."*

C **Code**

Exit /plan → generate

→ *"Plan approved. Now generate the login view first — just the APIView class and the URL config in urls.py."*

T **Test**

Run & iterate

→ *"Run python manage.py test. Here's the error: [AttributeError: 'AnonymousUser' object has no attribute 'id']. Fix it and re-run."*

Each step builds on the previous — context informs the plan, the plan drives the code, the test closes the loop.

Let's Practice — Your Turn

Scenario:

You are building a To-Do List REST API. Add a feature that allows users to mark a task as complete.

P Plan

What would you type in /plan mode?

(task description + context: stack, conventions, constraints)

A Approve

What would you check in the plan?

(steps, assumptions, anything missing or wrong?)

C Code

How would you ask for the code?

(which file first? scope? one function or full feature?)

T Test

What would you test first?

(what command to run? what if it fails — what do you do?)

Debrief questions:

Did you use /plan mode? Was your context specific? Did you actually approve before coding? What would you test first?

MODULE 01

Claude Code Extension

Claude.md, skills, commands, and agents

Internal Use Only – Do Not Share Outside the Organization

What Belongs in CLAUDE.md



Stack & Versions

Node.js 20, TypeScript 5, Fastify (NOT Express), Prisma 5, Vitest (NOT Jest)



Project Structure

src/routes/ for handlers · src/services/ for logic · tests/ mirrors src/



Commands

npm run test · npm run lint · npm run build — so Claude can verify its work



Coding Conventions

JSDoc on all exports · no `any` types · services must not import from routes



Anti-Patterns (Do NOT)

Never use Express. Never console.log. Never modify schema directly.



Key References

API contract: /docs/openapi.yaml · DB schema: /prisma/schema.prisma

CLAUDE.md — Real Project Example

CLAUDE.md

```
# CLAUDE.md

## Stack
- Python 3.12 + Django 4.2 LTS
- Django REST Framework 3.15
- PostgreSQL 15 via Django ORM
- pytest-django (NOT unittest)

## Project Structure
apps/auth/    views · serializers
apps/core/    exceptions · utils
tests/        mirror apps/ layout

## Commands
python manage.py test   # run all
python manage.py migrate
ruff check .  # linter

## Conventions
- All views use DRF class-based
- Zod-style Serializer validation
- services/ must not import views/

## Do NOT
- Never use print() — use logger
- Never raw SQL — Django ORM only
- Never pip install without asking
```

Why each section matters

`## Stack`	Claude stops guessing your tech — exact versions prevent wrong assumptions
`## Commands`	Claude runs tests & lint after every codegen cycle to self-verify its work
`## Conventions`	Enforces team patterns automatically — no manual reminders every session
`## Do NOT`	Prevents known bad choices. Claude respects explicit exclusions every time

Pro tips

`Keep it concise`	Every line costs tokens on every session — only include what Claude needs
`Version-control`	Commit CLAUDE.md alongside code — it's as important as your linter config
`Update it often`	Add a section whenever Claude makes the same wrong assumption twice

Monorepo hierarchy

`repo/CLAUDE.md`	Global: company-wide rules, shared tooling
`apps/api/CLAUDE.md`	API-specific: Django patterns, ORM rules
`apps/frontend/CLAUDE.md`	Frontend: React, Tailwind, component structure

CLAUDE.md is loaded automatically at session start — every line applies to every task in the project.

MODULE 01

Claude Skills

SKILL.md files that encode best practices — so Claude reads them once and applies them every time.

Internal Use Only – Do Not Share Outside the Organization

Skills — Reusable Task Expertise

What Is a Skill?

A SKILL.md file in .claude/skills/ with curated instructions. Claude reads the relevant skill before starting that task type — producing higher quality without re-explaining best practices each time.

Public Skills
/mnt/skills/public/ — platform-provided (docx, pptx, pdf...)

User Skills
/mnt/skills/user/ — your custom workflows

Example Skills
/mnt/skills/examples/ — reference implementations

```
---
name: devcard-description
description:
  Generate a developer profile card as a styled HTML file.
  Trigger whenever someone says "make me a dev card", "generate my developer card",
  "create a dev card for [name]", or "show me a dev card". Ask for name, role,
  and top 3 skills if not provided. Output a single beautiful HTML file.
---

# Dev Card Generator

Creates a stylish **developer profile card** as a single HTML file.

## What to Ask the User

If not already provided, ask for:
1. **Name** — e.g. Maria Santos
2. **Role** — e.g. Backend Developer
3. **Top 3 skills** — e.g. Python, FastAPI, Docker

That's it. Generate immediately once you have those three things.

## Output

A single `dev-card.html` file with:
- Name + role
- Skill badges
- A short auto-generated fun bio (one sentence, make it witty)
- Dark card design, ready to screenshot and share

## Example Prompt That Triggers This Skill

> "Make me a dev card. I'm Ana Reyes, Full Stack Developer, skills: React, Node, PostgreSQL."

Generate the HTML file and present it to the user right away.
```

Skills encode expertise once. Every future task of that type benefits automatically.

MODULE 01

Claude Commands

Built-in slash commands + custom commands that automate recurring dev workflows.

Internal Use Only – Do Not Share Outside the Organization

Built-in Commands & Custom Commands

Built-in Slash Commands

`/clear`	Clear conversation history and free up context
`/compact`	Clear history, keep summary. Accepts: /compact [hint]
`/cost`	Show total cost and duration of session
`/doctor`	Diagnose and verify installation and settings
`/memory`	Edit Claude memory files
`/status`	Show version, model, account, API connectivity
`/diff`	View uncommitted changes and per-turn diffs
`/commit`	Create a git commit from staged changes
`/review`	Review a pull request
`/security-review`	Security review of pending branch changes

Custom Commands

Create .claude/commands/my-command.md
Type /my-command to execute it.

```
# PR Review Checklist
# Usage: /pr-review

Review staged changes:

- No console.log in code
- All new functions have JSDoc
- Tests exist for new logic
- No hardcoded secrets
- Error paths use AppError
- No TypeScript `any` types

Output ✓/✗ per item with file+line.
```

Commands are version-controlled — they ship with the repo and improve over time.

MODULE 01

Claude Agents & MCP

Autonomous multi-step workflows — and connecting Claude to external tools via MCP.

Internal Use Only – Do Not Share Outside the Organization

Agentic Loop & Model Context Protocol

The Agentic Loop



Goal Given
Developer gives high-level task

Agent Plans
Claude generates implementation plan

Agent Acts
Writes code, runs tests, fixes failures

Observe
Reads output, checks against criteria

Done / Ask
Criteria met, or needs human input

⚠ **Safe practices:** Review plan before execution · Use git as safety net · Set explicit scope limits in your spec

MCP — External Tools

Model Context Protocol connects agents to external services:

Atlassian/Jira
Read/write tickets

Linear
Issues & sprints

GitHub
PRs, issues, repos

Figma
Design files

Notion
Pages & databases

HubSpot
CRM records

MODULE 02

Prompt Engineering

From prompt basics and anatomy to input-based techniques, reasoning methods, tone & style, optimization, and prompt templates.

- What is a Prompt?
- Input-Based Techniques
- Reasoning Techniques
- Tone & Style
- Prompt Optimization
- Prompt Templates

Internal Use Only – Do Not Share Outside the Organization

What Is a Prompt? — And Prompt Engineering?

What is a Prompt?

A prompt is any input you send to an LLM — a question, instruction, code snippet, or combination of all three. It is the primary way you communicate intent to the model.

What is Prompt Engineering?

The practice of designing and refining inputs to LLMs to reliably produce accurate, relevant, and high-quality outputs. It's the skill of making the model do exactly what you need.

Python Dev Example

❌ Fix the bug

✅
```
Fix the KeyError in user_service.py line 42
when user_id is not found in the cache dict.
```

⚡ **Same model, wildly different results**

Phrasing changes everything — not the model.

🐍 **Every Claude Code prompt you type**

Your /plan context, CLAUDE.md, custom commands.

🔧 **System prompts & API calls**

When building AI features into your Python apps.

📋 **Spec writing for Claude**

Telling Claude exactly what to build & how.

Internal Use Only – Do Not Share Outside the Organization

LLM Basics — How Models Process Prompts

Key insight: LLMs are next-token predictors. They don't 'understand' your intent — they predict the most statistically likely continuation of your input. Good prompts make the desired output the most likely prediction.

1

Tokenisation

Your text splits into tokens (sub-word fragments). Context limits are measured in tokens, not words.

2

Context window

Full conversation — system prompt + history + your message — loaded as one token sequence.

3

Prediction loop

Model samples next most-likely token repeatedly until stop condition or max token limit.

4

Temperature

Controls randomness. 0 = deterministic. 1+ = creative. Claude Code uses 1 by default.

What this means for your prompts

- Longer context = higher cost + risk of degraded quality
- Earlier tokens carry more weight — put important context first
- The model can't ask for clarification unless you explicitly let it
- Every token you provide reduces tokens available for output

Internal Use Only – Do Not Share Outside the Organization

Prompt Elements — The 4-Part Anatomy

Not every prompt needs all four elements — but knowing them lets you deliberately include or exclude each one.

01 Instruction

✅ `Summarise the following Python function in one sentence, focusing on what it returns.`

❌ `Summarise this.`

Be specific about format, scope, and output length.

02 Context

✅ `You are reviewing a Django REST API. The codebase uses DRF serializers for validation.`

❌ `Review this code.`

Context shifts the model's frame of reference before it starts predicting.

03 Input

✅ `Here is the function: def get_user(id): ...`

❌ `(No code provided)`

The specific content the instruction should be applied to.

04 Output

✅ `Return a JSON object: {summary: str, returns: str, raises: list}`

❌ `(No format specified)`

A signal that shapes the format, length, or style of the response.

Internal Use Only – Do Not Share Outside the Organization

Input-Based Prompt Techniques

Zero-Shot · Few-Shot · Prompt Chaining — with Python examples.

1.1 Zero-Shot

No examples — rely on pre-trained knowledge

1.2 Few-Shot

2–5 worked examples before the actual task

1.3 Prompt Chaining

Break complex tasks into sequential steps

Internal Use Only – Do Not Share Outside the Organization

1.1 Zero-Shot Prompting

Zero-shot: You give the model a task with no examples. You rely entirely on the model's pre-trained knowledge. Simplest and fastest — often sufficient for well-scoped tasks.

✓ Task is well-defined

Model already knows how to do this type of task well

✓ Low-stakes / exploratory

Testing or exploring — not in production

✓ Output format is simple

Short answer, classification, or translation

✓ No examples ready

Start here — add examples only if output is poor

Python examples

```
# Classification
Classify the following code review comment as:
'bug', 'style', 'performance', or 'question'.

Comment: 'This will throw if items is None.'
```

```
# Code explanation
Explain what this Python decorator does
in one sentence, for a junior developer:

@functools.lru_cache(maxsize=128)
def compute_hash(value: str) -> str: ...
```

⚠ Limitation: Unreliable for complex tasks, domain-specific formats, or nuanced instructions. The model defaults to its own style.

Internal Use Only – Do Not Share Outside the Organization

1.2 Few-Shot Prompting

Few-shot: Include 2–5 worked examples before the actual task. Examples show the model exactly what input → output looks like for your use case. More reliable than zero-shot for specific formats or domain tasks.

✓ **Use when zero-shot gives wrong format**

Model keeps formatting output differently than you need.

✓ **For domain-specific output**

Custom error messages, JSON shapes, naming conventions.

✓ **Examples trump description**

One good example = more precise than a paragraph of instructions.

! **Order matters**

Put the best/most representative example last — closest to the task.

! **Diverse examples are stronger**

Cover edge cases. Similar examples cause over-generalisation.

× **Don't use incorrect examples**

Wrong examples actively hurt output. Verify yours are correct.

Python example — Django error classifier

```
Classify the error type.

Error: "DoesNotExist: User matching..."
Type: Database

Error: "PermissionDenied: ..."
Type: Auth

Error: "ValidationError: field required"
Type: Input

# Now the real task:
Error: "IntegrityError: duplicate key..."
Type: ???

→ Claude: Type: Database  (follows pattern)
```

Internal Use Only – Do Not Share Outside the Organization

1.3 Prompt Chaining

Prompt chaining: Break a complex task into a sequence of smaller prompts where the output of each step becomes the input of the next. Ideal for multi-step workflows that exceed a single prompt's reliable scope.

1

Read & summarise

```
Read views.py.
Summarise the auth flow
in bullet points.
```

2

Identify issues

```
Given this summary,
list security issues
in the auth flow.
```

3

Write fix plan

```
For each issue listed,
write a fix plan
before touching code.
```

4

Implement

```
Implement fix #1
from the plan.
One file at a time.
```

- Output of step N feeds into step N+1
- Human checkpoint between steps = PACT Approve step
- Each step stays within a manageable scope
- Use /compact between long chains to save context

Internal Use Only – Do Not Share Outside the Organization

Reasoning Techniques

Chain of Thought · Tree of Thought — making Claude think before it acts.

2.1 Chain of Thought

Step-by-step reasoning before answering

2.2 Tree of Thought

Explore multiple reasoning paths in parallel

Internal Use Only – Do Not Share Outside the Organization

2.1 Chain of Thought (CoT) Prompting

CoT: Prompt the model to reason step-by-step before giving a final answer. This dramatically improves performance on complex tasks — debugging, planning, architecture, multi-step changes.

Why it works

Generating intermediate reasoning tokens steers the model through a logical path before it commits to an answer. It can't 'skip ahead' to a plausible-but-wrong answer.

- Complex debugging — trace the call stack before identifying root cause
- Architecture planning — reason about trade-offs before proposing a design
- Multi-step refactoring — think through dependencies before touching code
- Ambiguous requirements — reason through what the spec might mean first

Two ways to trigger CoT

```
# Zero-shot CoT — trigger phrase
Before writing any code, think step
by step about what could cause the
login() function to return None when
valid credentials are provided.
Then fix the root cause.
```

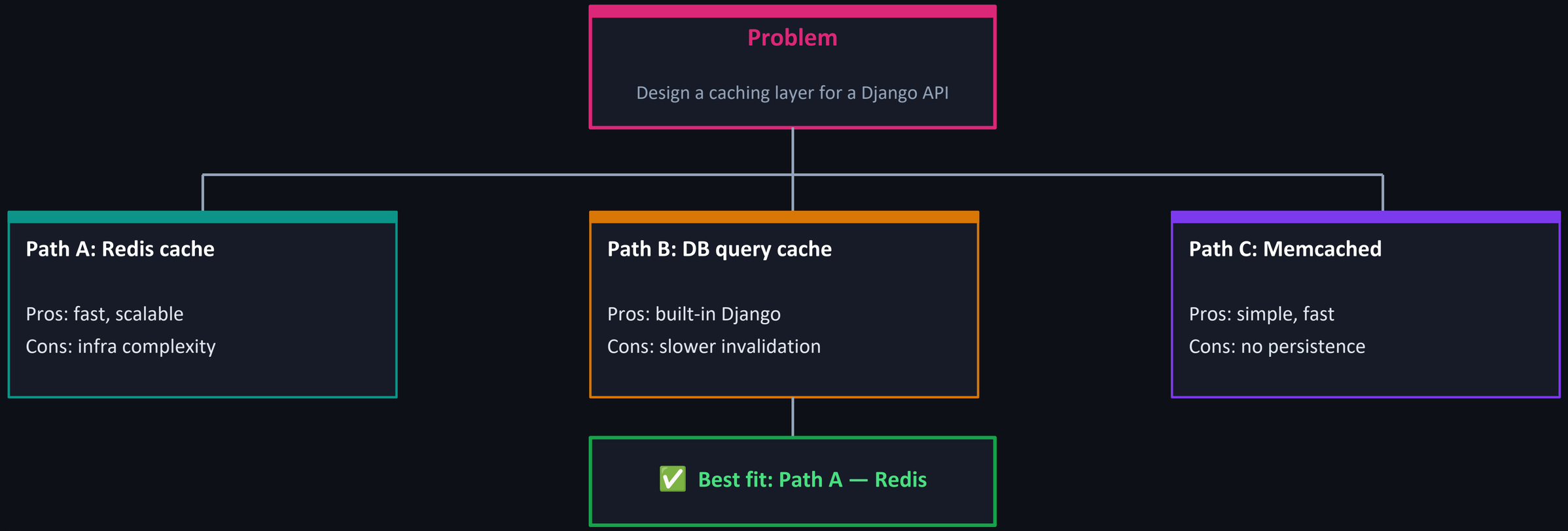
```
# Few-shot CoT — show reasoning
Q: Is 'items or []' safe if items=0?
Thinking: items=0 is falsy → 'or []'
  returns [] instead of 0 → data loss.
A: No. Use 'items if items is not None'

Q: Is 'value and value.strip()'
   safe if value='' ?
Thinking: value='' is falsy → short-
  circuits → returns ''. Safe here.
A: Yes, but use 'value is not None'.
```

Internal Use Only – Do Not Share Outside the Organization

2.2 Tree of Thought (ToT) Prompting

ToT: Instead of one reasoning path, ask the model to explore multiple approaches in parallel, evaluate each, and select the best. Useful for architecture decisions, algorithm selection, and open-ended design problems.




```
# ToT Python prompt
I need to add rate limiting to my FastAPI app.
Explore 3 approaches: A) Redis+slowapi  B) In-memory dict+TTL  C) DB counter
For each: pros, cons, complexity. Then recommend the best for < 1000 req/day.
```

MODULE 02 · ACTIVITY

Prompt Chaining — Add Auth to Your To-Do App

MODULE 01 · YOUR EXISTING APP

📋 To-Do List REST API

Feature already built: **Mark a task as complete**

React · Django · Spring Boot · Express · Laravel · any framework

MODULE 02 · NEW FEATURE TO ADD

🔐 Authentication Screens

Add to your app: **Login · Sign Up · Forgot Password**

No email sending required · UI + logic flow only · Use Prompt Chaining

1 Understand

READ & SUMMARISE

- Tell Claude about your existing app and its current stack
- Ask it to summarise what each of the 3 screens needs to do
- Keep it high-level — you want understanding, not code

🚫 No code in this step.

2 Map the Files

IDENTIFY CHANGES

- Pass the Step 1 summary as context into this prompt
- Ask Claude which files need to be created or modified
- Ask for a one-line description per file on what needs to change

🚫 No code in this step.

3 Write the Plan

PLAN · STOP & APPROVE ✋

- Pass the file list as context and ask for a step-by-step plan per file
- Read the plan — does it fit your app? Is anything missing?
- Correct Claude if needed, then explicitly approve before moving on

⚠️ STOP. Review and approve the plan before proceeding to Step 4.

4 Implement

ONE FILE AT A TIME

- Tell Claude the plan is approved and ask it to implement one file first
- Review each file before asking Claude to move to the next one
- Repeat until all 3 screens and supporting files are complete

✓ Login · ✓ Sign Up · ✓ Forgot Password

Internal Use Only – Do Not Share Outside the Organization

🔗 output of each step = input of next

✋ always stop and approve at Step 3

📁 one file at a time in Step 4

⏰ 15 min

Best Practices — Crafting Effective Prompts

		✅	❌
1	Write for your stack	`Write clean Python 3.12. Use type hints on all functions. No bare except.`	`Write good code`
2	Be specific and descriptive	`Explain what get_user() in src/db.py returns when id is None. One sentence.`	`Explain this`
3	Avoid negatives — use positives	`Write a pytest test for UserService.create(). Cover: 201, 400, 422.`	`Write a test`
4	Request structured output explicitly	`Return JSON: {function: str, returns: str, raises: list[str]}`	`Make it better`
5	Iterate — prompts are code	`Refactor auth.py for readability. Keep same signatures. No new dependencies.`	`Fix it`

Internal Use Only – Do Not Share Outside the Organization

Prompt Templates

Internal Use Only – Do Not Share Outside the Organization

Prompt Template 1 — RTF → RTFC → RTCIF

Templates evolve as your needs become more complex. Each level adds more structure and control over Claude's output.

RTF

Role · Task · Format

```
Role: You are a senior Python
developer.
Task: Review this Django view
for
  security vulnerabilities.
Format: Bullet list, one issue
per line.
```

Good starting point for most tasks.

RTFC

Role · Task · Format · Constraints

```
Role: You are a senior Python
developer.
Task: Review this Django view
for
  security vulnerabilities.
Format: Bullet list, one issue
per line.
Constraints: Focus only on auth
and
  input validation. Max 5
issues.
```

Adds constraints to control scope & length.

RTCIF

Full Template

```
Role: Senior Python developer.
Task: Review the Django view
below
  for security vulnerabilities.
Context: DRF app, JWT auth,
Postgres.
Input: [paste views.py here]
Format: Markdown table — Issue |
  Severity | Fix
Constraints: Auth + validation
only.
```

Production-grade. Use for critical tasks.

Internal Use Only – Do Not Share Outside the Organization

Other Templates

Internal Use Only – Do Not Share Outside the Organization

CO-STAR Framework

CO-STAR provides a complete context-setting structure. Originally from Singapore Government Tech Agency. Excellent for complex tasks where context heavily shapes the output.

C — Context

Background on the situation

```
You are reviewing a FastAPI app deployed on AWS Lambda.
```

O — Objective

What you want Claude to do

```
Identify all functions that could cause cold-start latency issues.
```

S — Style

Communication style

```
Write like a senior backend engineer reviewing a PR.
```

T — Tone

Attitude/emotion

```
Direct and concise. No filler. Flag critical issues clearly.
```

A — Audience

Who will read the response

```
Mid-level Python developer who knows Django but is new to serverless.
```

R — Response

Desired format and length

```
Markdown list. Issue name | Why it causes latency | Fix.
```

Internal Use Only – Do Not Share Outside the Organization

AUTOMAT Framework

AUTOMAT is designed for automation and pipeline prompts. Ideal when Claude is performing an autonomous or semi-autonomous task in a workflow — not just answering a question.

A	**Act as**	`Act as a Python code reviewer specialised in Django REST APIs.`
U	**User context**	`User is a mid-level Python dev building their first production Django app.`
T	**Task**	`Review the serializer in serializers.py for validation issues.`
O	**Output format**	`JSON: [{field, issue, severity: low/med/high, fix}]`
M	**Mode/style**	`Be direct. Skip praise. Only flag real issues.`
A	**Audience**	`Developer who will apply fixes immediately — no explanation needed.`
T	**Temperature cue**	`Be consistent and deterministic — this feeds into a CI pipeline.`

BRAIN Framework (Enterprise Research)

BRAIN is designed for enterprise research tasks — when you need Claude to deeply analyze a problem, not just respond to it. Especially useful for architecture decisions and technical investigations.

B	**Background** Set the full context upfront	`We are building a high-traffic Django API (50k req/day). Currently using session auth. Planning to migrate to JWT.`
R	**Request** What you need Claude to do	`Research and compare: SimpleJWT vs Djoser vs custom JWT implementation for our use case.`
A	**Action steps** How Claude should approach it	`For each option: 1) explain how it works, 2) list pros/cons, 3) estimate migration complexity.`
I	**Input data** Data Claude should analyse	`[paste current auth.py and settings.py here]`
N	**Notes/constraints** Guardrails and preferences	`We use PostgreSQL, cannot add Redis. Team has 2 Python devs. Zero downtime migration required.`

Goal–Input–Output (GIO)

GIO is the most practical template for real-world automation, proposals, and pipelines. Every component has a job — and nothing is left ambiguous.

GOAL	**What you want** `Generate a Django REST API endpoint that creates a new User and returns a JWT token on success.`
INPUTS	**Data provided to Claude** `User model fields: username, email, password. Serializer: UserCreateSerializer. Auth: SimpleJWT.`
CONSTRAINTS	**Rules Claude must follow** `Use DRF class-based views. Follow PEP8. No print() — use logging. Password must be hashed via make_password().`
OUTPUT	**Format and structure** `Return: views.py (APIView class), serializers.py update, urls.py entry. Each in a separate code block.`
VERIFY	**How to check quality** `After generating, run: python manage.py test users.tests.test_register — all tests must pass.`

Techniques vs Templates — Why You Need Both

Techniques and Templates work together — they solve different problems. Picking the right technique decides HOW Claude thinks. Picking the right template decides HOW you structure your ask.

TECHNIQUE — How Claude thinks

Controls the reasoning pattern Claude uses to arrive at an answer.

Zero-Shot	No examples — rely on training knowledge
Few-Shot	Pattern from examples you provide
Prompt Chaining	Break into sequential steps
Chain of Thought	Force step-by-step reasoning
Tree of Thought	Explore multiple paths in parallel

+

TEMPLATE — How you structure the ask

Controls the format and completeness of your prompt input.

RTF / RTFC / RTCIF	Role, task, format — escalating detail
CO-STAR	Full context-setting for complex tasks
AUTOMAT	Automation and pipeline workflows
BRAIN	Enterprise research and analysis
Goal–Input–Output	Proposals, automation, real-world tasks

Rule: Pick your TECHNIQUE first (how Claude reasons), then wrap it in a TEMPLATE (how you structure the ask). Used together = maximum output quality.

Prompt Optimization — Why It Matters

Prompt optimization is the systematic process of refining prompts to maximize output quality, consistency, and efficiency. The same task — different structure = dramatically different results.

🎯 Consistency

Same prompt structure = predictable, repeatable outputs across sessions and team members.

⚡ Efficiency

Well-optimized prompts use fewer tokens — less cost, less context bloat, faster responses.

🏗️ Scalability

Optimized prompts work reliably at scale — in pipelines, automation, and production apps.

🤝 Collaboration

Shared prompt templates align teams — everyone gets the same quality output from Claude.

Internal Use Only – Do Not Share Outside the Organization

MODULE 02

Writing Specifications

Structured specs consistently outperform even perfect prompts for complex, multi-file tasks.

Internal Use Only – Do Not Share Outside the Organization

What Makes a Good Spec? The 5 Categories

1. Task Definition

- Clear action verb (build, fix, refactor, add, remove)
- Scope boundaries — what's in / what's out
- Context on WHY (the problem being solved)
- Single coherent task (not multiple bundled)

```
"Refactor login to JWT. Don't touch registration."
```

2. Technical Context

- Stack / framework / versions
- File locations to create or modify
- Dependencies to use (or avoid)
- Patterns to follow from existing code

```
"Use Prisma in src/repositories/. Follow userRepo pattern."
```

3. Expected Output

- Deliverable format (files, functions, endpoints)
- Behaviour description (what happens when it works)
- Interface / API shape (signatures, params, returns)
- Example usage (how it will be called)

```
"parseCSV(path: string): Promise<Record[]> in utils/"
```

4. Constraints

- Must-haves (non-negotiable requirements)
- Must-NOT-haves (things to avoid)
- Performance limits (size, speed, memory)
- Compatibility (Node version, browser support)

```
"Max 100MB, no new deps, Node 18+, don't modify tests"
```

5. Edge Cases & Errors

- Invalid input handling (what to do with bad data)
- Empty / null cases (behaviour for missing data)
- Failure modes (throw vs return null vs log)
- Boundary conditions (min/max, limits)

```
"Throw ValidationError for X. Return null if not found."
```

Writing Good Specs — Language, Pitfalls & Pre-flight Check

⚡ Power Words

Good action verbs:

Build · Create · Add · Implement · Refactor · Fix · Remove · Replace · Migrate · Extract · Optimise · Update · Convert

Good constraint phrases:

`"Don't modify…"` · `"Must not exceed…"` · `"Keep existing…"`

`"Follow the pattern in…"` · `"Return X when…"` · `"Throw if…"`

⚠ Avoid These

✕ `"Make it better"`	→ Better how? Be specific
✕ `"Handle errors"`	→ Which errors? What behaviour?
✕ `"Add database"`	→ Which one? What ORM?
✕ `"Make it fast"`	→ Define a measurable target
✕ `"Clean up code"`	→ Which files? What pattern?

✈ PRE-FLIGHT CHECKLIST — Ask yourself before submitting the spec

- ☐ Would someone else build the SAME thing from this spec?
- ☐ Did I specify what 'DONE' looks like?
- ☐ Did I mention what NOT to do?
- ☐ Did I cover the unhappy paths (errors, edge cases)?
- ☐ Did I provide enough context about existing code?
- ☐ Is this ONE task, or should I split it up?

Internal Use Only – Do Not Share Outside the Organization

SPECIFICATION EXAMPLE · DJANGO PYTHON

User Authentication System

MyApp API · Module 02 Spec Example



Django 4.2 LTS · DRF · Python 3.12 · PostgreSQL · PyJWT · argon2-cffi · pytest-django

1

Task Definition

What we're building:
JWT-based authentication for the MyApp Django REST API — register, login, token-protected endpoints, and refresh/logout.

Why now:
Current API has no authentication. All DRF endpoints are publicly accessible — a hard blocker before beta launch.

Success looks like:
Users can register, log in, receive a short-lived access token, and use it to access protected views.

2

Technical Context

Runtime	`Python 3.12 (run with python -X gil=0)`
Framework	`Django 4.2 LTS + DRF 3.15`
Database	`PostgreSQL 15 via Django ORM`
Auth tokens	`PyJWT 2.x (already in requirements.txt)`
Passwords	`argon2-cffi via django[argon2]`
Validation	`DRF Serializers — follow pattern in users/serializers.py`
Errors	`Custom exception handler in core/exceptions.py`
New files	`apps/auth/views.py · serializers.py apps/auth/models.py · urls.py`
Env vars	`JWT_SECRET, JWT_EXPIRY, REFRESH_TOKEN_EXPIRY`
Tests	`pytest-django → tests/auth/test_views.py`

3 Functional Requirements

3

1 **POST /api/auth/register/**

Accepts { email, password, password_confirm } — validates and creates user via UserSerializer

2 **Password hashing**

Argon2id via PASSWORD_HASHERS = ['django.contrib.auth.hashers.Argon2PasswordHasher']

3 **Register 201 response**

Returns { id, email, created_at } — no password field. DRF SerializerMethodField excludes it

4 **Duplicate email**

Returns 400 with { email: ['A user with that email already exists.'] } (DRF default)

5 **POST /api/auth/login/**

Accepts { email, password } — authenticates via authenticate() — returns access + refresh JWT

6 **Refresh token**

Refresh token stored in RefreshToken model (DB) with 7d expiry — indexed on token hash

7 **POST /api/auth/token/refresh/**

Accepts { refresh } — validates against DB — returns new access token (1h expiry from JWT_EXPIRY)

8 **POST /api/auth/logout/**

Accepts { refresh } — deletes RefreshToken row from DB — invalidates that session

9 **Protected views**

JWTAuthentication class in DEFAULT_AUTHENTICATION_CLASSES — returns 401 for invalid/missing tokens

10 **Invalid credentials**

Returns 401 { detail: 'Invalid credentials.' } — same message for wrong email and wrong password

4 Non-Functional Requirements · 5 Out of Scope

4 Non-Functional Requirements

Security
Passwords never in logs, responses, or tracebacks. Use Django's sensitive_variables decorator.

Secrets
JWT_SECRET from os.environ — raise ImproperlyConfigured at startup if missing.

Validation
All inputs validated with DRF Serializers before any ORM call. Invalid input returns 400 with field-level errors.

Performance
Token validation must NOT hit the DB — validate JWT signature only in JWTAuthentication.authenticate().

Latency
Login p95 < 600ms — Argon2id is the bottleneck; memory_cost=65536, time_cost=2 is the agreed setting.

Errors
All API errors via custom exception handler — no raw Django 500 pages or unhandled exceptions.

5 Out of Scope

Explicitly excluded from this ticket:

- x **Django allauth / social login**
  Google, GitHub — planned Phase 2
- x **django-otp / 2FA**
  Separate ticket, not this sprint
- x **Email verification**
  Deferred — users trusted for now
- x **Password reset views**
  Separate ticket — future sprint
- x **Throttling on login**
  Handled by nginx rate limiting layer
- x **Frontend / templates**
  DRF API only — no HTML views in this ticket

6 Acceptance Criteria

6

Definition of done — Claude self-checks against every item before considering the task complete.

- v **POST /api/auth/register/ (valid)**
  Returns 201 with {id, email, created_at} — no password in response body
- v **POST /api/auth/register/ (duplicate)**
  Returns 400 with field-level email error from DRF
- v **POST /api/auth/login/ (valid)**
  Returns 200 with access and refresh JWT tokens
- v **POST /api/auth/login/ (wrong password)**
  Returns 401 with generic 'Invalid credentials.' — no enumeration
- v **Protected view — valid token**
  Returns 200 with expected data payload
- v **Protected view — no token**
  Returns 401 { detail: 'Authentication credentials were not provided.' }
- v **Protected view — expired token**
  Returns 401 { detail: 'Token has expired.' }
- v **POST /api/auth/token/refresh/ (valid)**
  Returns new access token; refresh token entry still in DB
- v **POST /api/auth/logout/**
  Deletes RefreshToken row; subsequent /token/refresh/ returns 401
- v **Test coverage**
  pytest tests cover happy path + primary error case for each endpoint
- v **Password exposure**
  No password appears in test fixtures, Django logs, or any API response

MODULE 3

Claude Code in Our SDLC

How we've operationalised Claude Code as a structured, auditable six-phase workflow.

Internal Use Only – Do Not Share Outside the Organization

The Six-Phase SDLC Workflow

1 Score Specs

Score ≥ 80 required

score-specs skill evaluates spec quality. No code until threshold is met.

2 Validate

No duplicate work

specs-code-relevance-audit classifies every requirement vs. existing code.

3 Plan

Explicit approval required

Claude presents implementation plan. No code generation until approved.

4 Build & Test

Tests pass every cycle

run-tests + test-web-application after every codegen cycle. No exceptions.

5 Push

Annotated PR created

push-to-github creates labeled PR. update-specs syncs spec document.

6 Review

Tech Lead reviews

pull-review fetches and categorizes PR comments for next iteration cycle.

Developer + Claude Code
AI-Assisted Development Workflow

Writing Specifications — Add Features to Your To-Do App

YOUR EXISTING APP

To-Do List REST API

✓ Mark as complete ✓ Login / Sign Up / Forgot Password

React · Django · Spring Boot · Express · any framework

MODULE 02 · NEW FEATURES TO SPEC

Task Categories & Tags · Due Dates

Write ONE combined specification **covering both features**

Use the 5-category spec framework you just learned

1 Task Definition

- Clear action verb
- Scope: what's in / out
- Why now (the problem)
- Single coherent task

"Add categories + due dates to tasks. No recurring tasks."

2 Technical Context

- Stack & versions
- File locations
- Dependencies to use
- Patterns to follow

"Follow TaskSerializer pattern in serializers.py"

3 Expected Output

- Deliverable format
- Behaviour when it works
- API / interface shape
- Example usage

"GET /api/tasks/?category=work returns filtered list"

4 Constraints

- Must-haves
- Must-NOT-haves
- Performance limits
- Compatibility rules

"No new deps. Tag name max 30 chars. Node 18+"

5 Edge Cases & Errors

- Invalid input handling
- Empty / null cases
- Failure modes
- Boundary conditions

"Throw 400 if due_date is in the past. Return [] if no tags."

Internal Use Only – Do Not Share Outside the Organization

✏️ write the spec yourself

✓ run the pre-flight checklist

🎯 target score ≥ 80 / 100

⏰ 15 min

MODULE 3

AI USAGE POLICY

Internal Use Only – Do Not Share Outside the Organization

MODULE 04

CAPSTONE

Everything comes together. Build a real feature end-to-end using the complete Claude Code workflow.

Internal Use Only – Do Not Share Outside the Organization

Capstone Lab — Rate Limiting Middleware

A

Environment Setup

Verify CLAUDE.md is current. Add middleware conventions, test framework, existing patterns.

B

Write the Specification

Complete 6-section spec for rate-limiting middleware. Save as specs/rate-limiting.md.

C

Prompt-Driven Planning

Ask Claude to read spec and present an implementation plan before any code is written.

D

Iterative Development

Review each generated file. Use follow-up prompts to correct deviations. Verify tests pass.

E

Run Custom Commands

Run /pr-review. Fix any flags. Run /security-review before pushing.

F

Push & Retrospective

Push PR with Claude Code. Answer the 5 retrospective questions. Document lessons learned.

Quick Reference Card

Model Strings

Opus:	`claude-opus-4-6`
Sonnet:	`claude-sonnet-4-6`
Haiku:	`claude-haiku-4-5-20251001`

Install

```
npm i -g @anthropic-ai/claude-code
```

Docs

`code.claude.com/docs`

`docs.anthropic.com/en/docs/claude-code`

7 Prompt Principles

- 01 Be Specific
- 02 Set a Role
- 03 Define Format
- 04 Use Examples
- 05 State Constraints
- 06 Step-by-Step
- 07 Iterate

6-Section Spec

- §1 Overview & Goal
- §2 Technical Context
- §3 Functional Reqs
- §4 Non-Functional
- §5 Out of Scope
- §6 Acceptance Criteria

1 Score | **2 Validate** | **3 Plan** | **4 Build** | **5 Push** | **6 Review**

You're Ready.

Now go build something.

- Apply the full workflow to a real feature this week
- Write and peer-review your team's CLAUDE.md
- Create 3 custom commands for your most repeated tasks
- Design a skill for your team's highest-value task type



 Internal Use Only – Do Not Share Outside the Organization